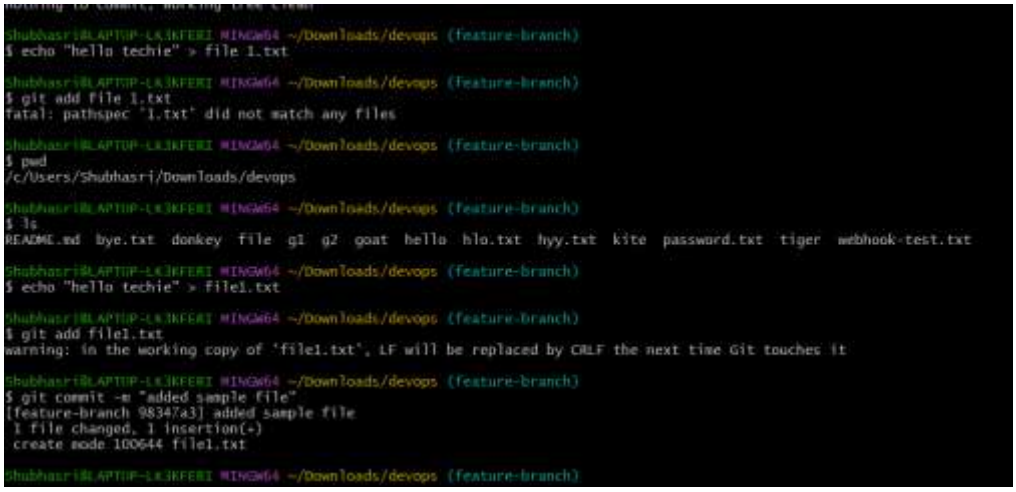
GIT AND GITHUB CHALLENGE

1. **Resolve Merge Conflicts**

- **Create a merge conflict intentionally (two users editing the same line).**
- **Resolve the conflict and push the changes.**



Steps :

1. **Create a new directory and navigate**
2. **Create an intentional merge conflict**
3. **Trigger the conflict by merging feature branch to main**
4. **Inspect and resolve the conflict by opening file2.txt**
5. **finalizes the merge and push**
6. **Git may show a merge conflict because both branches modified the same line.**
7. **Pushes the resolved file to the remote user1 branch on GitHub.**

Successfully created a merge conflict, resolved it, committed the changes, and pushed the resolved branch to GitHub.

```
Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (master)
$ git switch feature-branch
Switched to branch 'feature-branch'
Your branch is up to date with 'origin/feature-branch'.

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git merge master
Auto-merging .gitignore
Auto-merging hello
CONFLICT (content): Merge conflict in hello
Automatic merge failed; fix conflicts and then commit the result.

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch|MERGING)
$ git status
On branch feature-branch
Your branch is up to date with 'origin/feature-branch'.

You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Changes to be committed:
        modified:   .gitignore
        new file:   file2.txt

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   hello

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        file


Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch|MERGING)
$ notepad file1.txt


Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch|MERGING)
$ git status
On branch feature-branch
Your branch is up to date with 'origin/feature-branch'.

You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Changes to be committed:
        modified:   .gitignore
        new file:   file2.txt

Unmerged paths:
  (use "git add <file>..." to mark resolution)
```

```
Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch|MERGING)
$ git add file1.txt

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch|MERGING)
$ git commit -m "Resolve merge conflict"
error: Committing is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
hint: as appropriate to mark resolution and make a commit.
fatal: Exiting because of an unresolved conflict.
U       hello

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch|MERGING)
$ git push origin feature-branch
Everything up-to-date

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch|MERGING)
$ git status
On branch feature-branch
Your branch is up to date with 'origin/feature-branch'.

You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Changes to be committed:
        modified:   .gitignore
        new file:   file2.txt

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   hello

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        file


Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch|MERGING)
$ nano file2.txt

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch|MERGING)
$ git add file2.txt

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch|MERGING)
$ git commit -m "Resolve mergr conflict in hello"
error: Committing is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
hint: as appropriate to mark resolution and make a commit.
fatal: Exiting because of an unresolved conflict.
U       hello

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch|MERGING)
$ git commit -m "Resolve merge conflict in hello"
error: Committing is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
```

```
Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch|MERGING)
$ nano hello

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch|MERGING)
$ git add hello

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch|MERGING)
$ git commit -m "Resolve mergr conflict in hello"
[feature-branch b413b48] Resolve mergr conflict in hello

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git push origin feature-branch
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 472 bytes | 472.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:shubhasri015-hash/devops.git
   7abe123..b413b48  feature-branch -> feature-branch

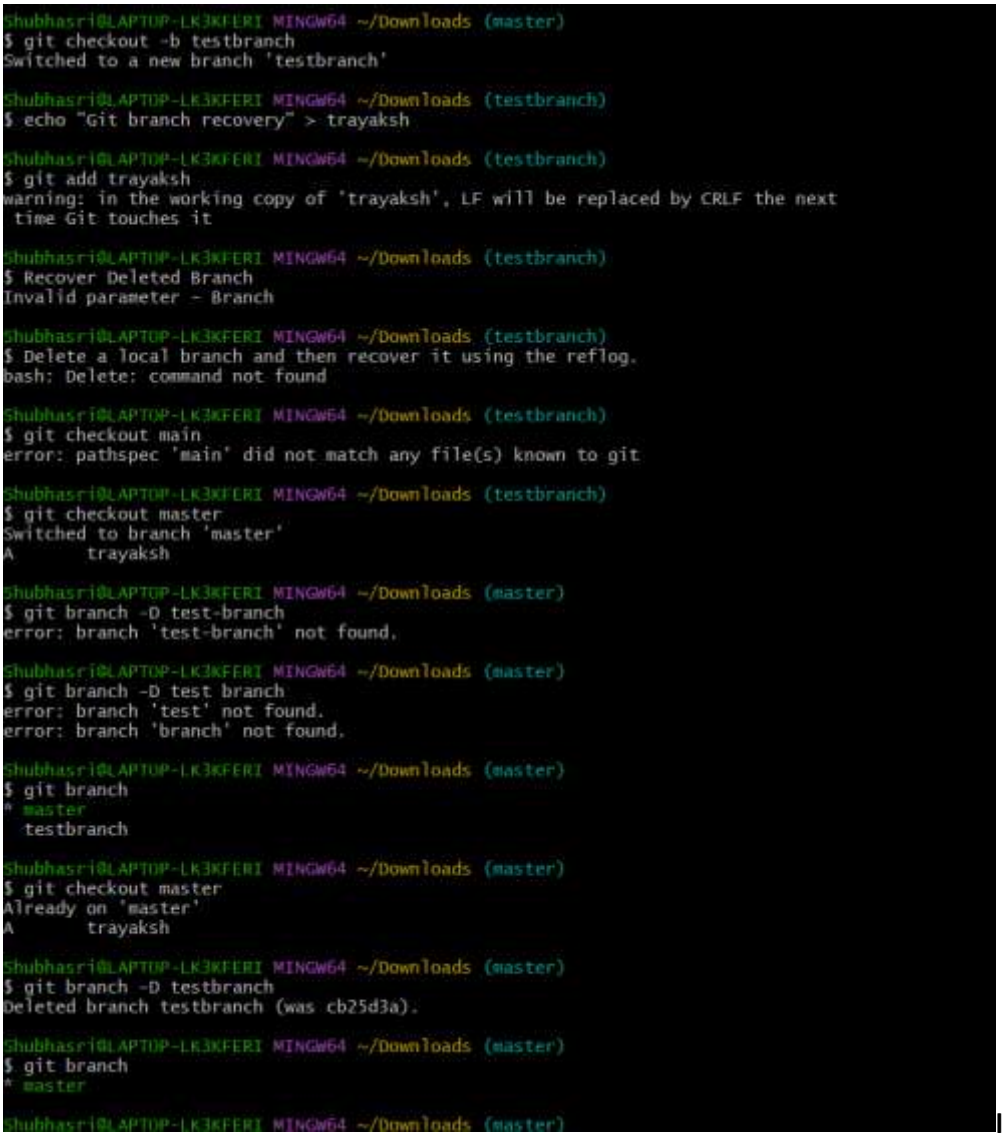
Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$
```

2. Recover Deleted Branch

- **Delete a local branch and then recover it using the reflog**

  Steps :

  1. **Created a new branch named test-branch.**
  2. **Create a commit**
  3. **Switched from test-branch to main**
  4. **Deleted the local test-branch (git branch -D test-branch)**
  5. **Checked the recent Git actions and identified the commit hash of the deleted branch**



```
Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads (master)
$ git reflog
cb25d3a (HEAD -> master) HEAD@{0}: checkout: moving from master to master
cb25d3a (HEAD -> master) HEAD@{1}: checkout: moving from testbranch to master
cb25d3a (HEAD -> master) HEAD@{2}: checkout: moving from master to testbranch
cb25d3a (HEAD -> master) HEAD@{3}: commit (initial): add sample file

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads (master)
$ git branch test-branch cb25d3a

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads (master)
$ git branch
* master
  test-branch

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads (master)
$ git checkout test-branch
Switched to branch 'test-branch'
A       trayaksh

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads (test-branch)
$
```

Checked the recent Git actions and identified the commit hash of the deleted branch.

Recreated the branch using the actual commit hash from the reflog.

Confirmed that the branch was recovered successfully.

3. Undo Wrong Push

- **Push a wrong commit to GitHub, then undo it without losing history.**

Steps:

1.

Create a wrong commit: git commit -m “sample one"

1. **Push to GitHub: git push origin master**
2. **Undo the commit: git revert HEAD**
3. **Push the revert commit: git push origin master**
4. **Check history: git log --oneline**

Commands meaning:

- **git revert HEAD — Undoes the latest commit without deleting history.**
- **git push — Uploads changes to GitHub.**

```
Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git log -5
commit b413b48d992637a7d692f07ed92dd1573eccaacc (HEAD -> feature-branch, origin/feature-branch, test-branch)
Merge: 7abe123 fd5b531
Author: shubhasri015-hash <shubhasri015@gmail.com>
Date:   Mon Oct 26 13:01:09 2026 +0530

    Resolve mergr conflict in hello

commit fd5b53102218d30321fa8dd99f23c9ece16e8a88 (master)
Author: shubhasri015-hash <shubhasri015@gmail.com>
Date:   Mon Oct 26 12:45:42 2026 +0530

    added sample file2

commit 7abe123cb15aad30d3f7f7af15cacb328cf18b04
Merge: 98347a3 74cbbb2
Author: shubhasri015-hash <shubhasri015@gmail.com>
Date:   Mon Oct 26 12:40:40 2026 +0530

    Merge branch 'feature-branch' of github.com:shubhasri015-hash/devops into feature-branch

commit 98347a3397f698de5847e4eab0ae03e938ea6b77
Author: shubhasri015-hash <shubhasri015@gmail.com>
Date:   Mon Oct 26 12:32:45 2026 +0530

    added sample file

commit e4775de45728e9088d1c759a446e9543d9ecacd2
Author: shubhasri015-hash <shubhasri015@gmail.com>
Date:   Sun Oct 25 21:31:09 2026 +0530

    Merged commits:feature updates and files

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ echo "sample one" > bomma

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git add bomma
warning: in the working copy of 'bomma', LF will be replaced by CRLF the next time Git touches it

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git commit -m "added sample one"
[feature-branch b1d970a] added sample one
 1 file changed, 1 insertion(+)
 create mode 100644 bomma

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git push -u origin master
Everything up-to-date
branch 'master' set up to track 'origin/master'.

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git push -u origin feature-branch
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
```

MINGW64:/c/Users/Shubhasri/Downloads/devops

```
Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git log -5
commit b1d970a128ff5eec79864e8041e6e9518725ebcb (HEAD -> feature-branch, or
Author: shubhasri015-hash <shubhasri015@gmail.com>
Date:   Mon Oct 26 15:02:16 2026 +0530

    added sample one

commit b413b48d992637a7d692f07ed92dd1573eccaacc (test-branch)
Merge: 7abe123 fd5b531
Author: shubhasri015-hash <shubhasri015@gmail.com>
Date:   Mon Oct 26 13:01:09 2026 +0530
```

4. Amend a Commit

- **Make a commit, then add a missing file to it using git commit --amend.**

Steps :

git commit --amend — Modifies the previous commit.

- **--no-edit — Keeps the existing commit message.**
- **git show --stat HEAD — Displays the files included in the latest commit.**
- **Result: Added the missing file to the previous commit without creating a separate commit.**
-

```
Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ echo "fourth file" > fourth

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git add fourth
warning: in the working copy of 'fourth', LF will be replaced by CRLF the next t
ime Git touches it

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git commit -m "added fourth file"
[feature-branch 8e1d19e] added fourth file
 2 files changed, 2 insertions(+)
 create mode 100644 file
 create mode 100644 fourth

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ echo "fifth file" > fifth

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git add fifth
warning: in the working copy of 'fifth', LF will be replaced by CRLF the next ti
me Git touches it

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git commit --amend --no-edit
[feature-branch f594517] added fourth file
 Date: Mon Oct 26 15:21:48 2026 +0530
 3 files changed, 3 insertions(+)
 create mode 100644 fifth
 create mode 100644 file
 create mode 100644 fourth

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git show --stat HEAD
commit f594517406a70db29842e65a5fc71a902cd6b3f8 (HEAD -> feature-branch)
Author: shubhasri015-hash <shubhasri015@gmail.com>
Date:   Mon Oct 26 15:21:48 2026 +0530

    added fourth file

 fifth  | 1 +
 file   | 1 +
 fourth | 1 +
 3 files changed, 3 insertions(+)

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$
```

5. **Cherry-pick a Commit. ◦ Take a specific commit from one branch and apply it to another branch..**

Steps :

- **git cherry-pick applies a specific commit from another branch to your current branch.**
- **git log --oneline displays commit history.**
- **git checkout switches branches.**

Result: Successfully applied a specific commit from the feature branch to the master branch.

```
Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature)
$ echo "Cherry-pick example" > uday

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature)
$ git add uday
warning: in the working copy of 'uday', LF will be replaced by CRLF the next tim
e Git touches it

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature)
$ git commit -m "added a file"
[feature 6bd8397] added a file
 1 file changed, 1 insertion(+)
 create mode 100644 uday

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature)
$ git log --oneline
6bd8397 (HEAD -> feature) added a file
f594517 (feature-branch) added fourth file
3ac1788 (origin/feature-branch) Revert "added sample one"
b1d970a added sample one
b413b48 (test-branch) Resolve mergr conflict in hello
fd5b531 (master) added sample file2
7abe123 Merge branch 'feature-branch' of github.com:shubhasri015-hash/devops int
o feature-branch
98347a3 added sample file
e4775de Merged commits:feature updates and files
302db89 added 3 files
8dc4a1a added two files
2a14382 added 1 file
06bdfbf updated commit message Successfully
74cbbb2 test github webhook
f110d6a donkey
4ce15cc Initial commit

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature)
$ git checkout feature
Already on 'feature'

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature)
$ git checkout feature-branch
Switched to branch 'feature-branch'
Your branch is ahead of 'origin/feature-branch' by 1 commit.
  (use "git push" to publish your local commits)

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git cherry-pick 6bd8397
[feature-branch a3bbf61] added a file
 Date: Mon Oct 26 15:29:43 2026 +0530
 1 file changed, 1 insertion(+)
 create mode 100644 uday

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git log --oneline
a3bbf61 (HEAD -> feature-branch) added a file
f594517 added fourth file
3ac1788 (origin/feature-branch) Revert "added sample one"
b1d970a added sample one
b413b48 (test-branch) Resolve mergr conflict in hello
fd5b531 (master) added sample file2
```

6. Interactive Rebase

Reorder and squash multiple commits into a single clean commit

- **Steps : git rebase -i — Opens interactive rebase mode.**
- **pick — Keeps a commit.**
- **squash — Combines a commit with the previous commit.**
- **git log --oneline — Displays commit history.**

Result: Successfully combined three commits into one clean commit.

```
Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ echo "first file" > rebase1

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git add rebase1
warning: in the working copy of 'rebase1', LF will be replaced by CRLF the next
time Git touches it

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git commit -m "add reabse1"
[feature-branch 88f4c0f] add reabse1
 1 file changed, 1 insertion(+)
 create mode 100644 rebase1

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ echo "second file" > rebase2

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git add rebase2
warning: in the working copy of 'rebase2', LF will be replaced by CRLF the next
time Git touches it

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git commit -m "add rebase2"
[feature-branch 646836f] add rebase2
 1 file changed, 1 insertion(+)
 create mode 100644 rebase2

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ echo "third file" > rebase3

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git add rebase3
warning: in the working copy of 'rebase3', LF will be replaced by CRLF the next
time Git touches it

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git commit -m "add rebase3"
[feature-branch ebc24f1] add rebase3
 1 file changed, 1 insertion(+)
 create mode 100644 rebase3

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git rebase -i HEAD~3
error: could not parse 'bc24f1'
error: invalid line 3: squash bc24f1 add rebase3
You can fix this with 'git rebase --edit-todo' and then run 'git rebase --contin
ue'.
Or you can abort the rebase with 'git rebase --abort'.

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch|REBASE)
$ git log --oneline
a3bbf61 (HEAD) added a file
f594517 added fourth file
3ac1788 (origin/feature-branch) Revert "added sample one"
b1d970a added sample one
```

```
error: invalid line 1: squash bc24f1 add rebase3

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch|REBASE)
$ git rebase --edit-todo

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch|REBASE)
$ git rebase --continue
[detached HEAD d58f178] add rebase1
 Date: Mon Oct 26 15:45:30 2026 +0530
 3 files changed, 3 insertions(+)
 create mode 100644 rebase1
 create mode 100644 rebase2
 create mode 100644 rebase3
Successfully rebased and updated refs/heads/feature-branch.

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git rebase --abort
fatal: No rebase in progress?

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git status
On branch feature-branch
Your branch is ahead of 'origin/feature-branch' by 3 commits.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git reflog
d58f178 (HEAD -> feature-branch) HEAD@{0}: rebase (continue) (finish): returning to refs/heads/feature-branch
d58f178 (HEAD -> feature-branch) HEAD@{1}: rebase (continue) (squash): add rebase1
6c88e67 HEAD@{2}: rebase (continue) (squash): # This is a combination of 2 commits.
88f4c0f HEAD@{3}: rebase: fast-forward
a3bbf61 HEAD@{4}: rebase (start): checkout HEAD~3
ebc24f1 HEAD@{5}: commit: add rebase3
646836f HEAD@{6}: commit: add rebase2
```

```
$ git rebase --edit-todo

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch|REBASE)
$ git rebase --continue
[detached HEAD e73141f] add reabse1
 Date: Mon Oct 26 15:45:30 2026 +0530
 3 files changed, 3 insertions(+)
 create mode 100644 rebase1
 create mode 100644 rebase2
 create mode 100644 rebase3
Successfully rebased and updated refs/heads/feature-branch.

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git log
commit e73141f0e175cdf8ce3b75d6911c85808a2f2bdd (HEAD -> feature-branch)
Author: shubhasri015-hash <shubhasri015@gmail.com>
Date:   Mon Oct 26 15:45:30 2026 +0530

    add reabse1

    add rebase2

    add rebase3

commit a3bbf6115d8aecc385279141f5d341798dcdbb1e
Author: shubhasri015-hash <shubhasri015@gmail.com>
Date:   Mon Oct 26 15:29:43 2026 +0530
```

```
    Merge branch 'feature-branch' of github.com:shubhasri015-hash/devops into feature-branch

commit 98347a3397f698de5847e4eab0ae03e938ea6b77

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git log --oneline -4
e73141f (HEAD -> feature-branch) add reabse1
a3bbf61 added a file
f594517 added fourth file
3ac1788 (origin/feature-branch) Revert "added sample one"

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ cat rebase1
first file
```

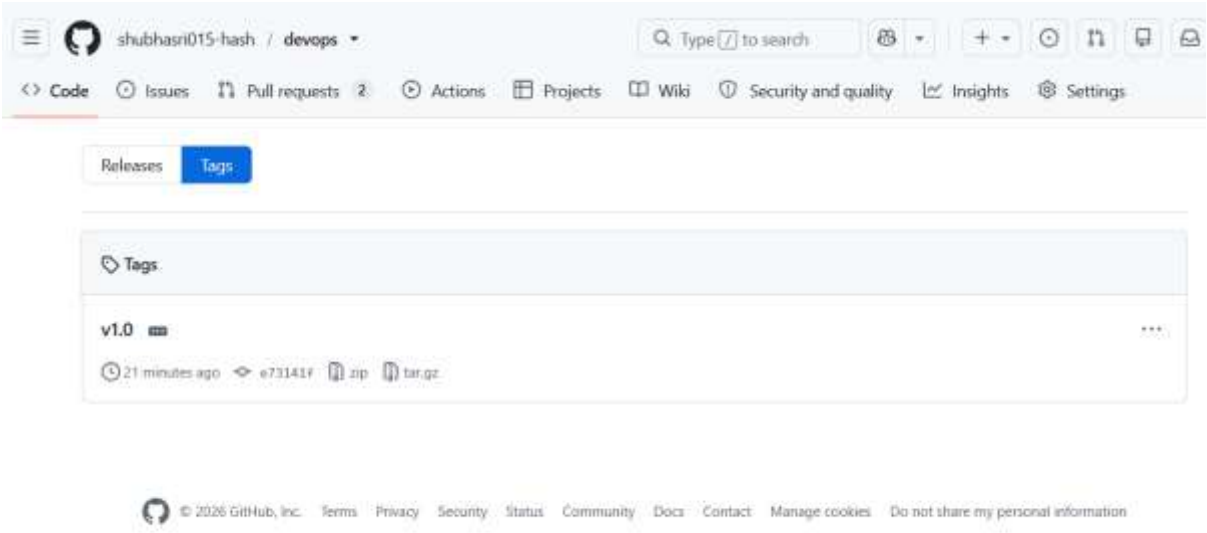
7. Tagging & Release

- **Create a version tag (v1.0), push it to GitHub, then delete and restore it.**

Steps :

- **git tag — Creates or lists tags.**
- **git push origin v1.0 — Pushes a tag to GitHub.**
- **git tag -d v1.0 — Deletes a local tag.**
- **git push origin --delete v1.0 — Deletes a remote tag.**

Result: Successful y created, pushed, deleted, and restored the v1.0 tag.



```
$ git tag v1.0

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git push origin v1.0
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 16 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (13/13), 972 bytes | 972.00 KiB/s, done.
Total 13 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), completed with 1 local object.
To github.com:shubhasri015-hash/devops.git
 * [new tag]         v1.0 -> v1.0

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git tag -d v1.0
Deleted tag 'v1.0' (was e73141f)

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git push origin --delete v1.0
To github.com:shubhasri015-hash/devops.git
 - [deleted]         v1.0

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git tag v1.0 .0
fatal: Failed to resolve '.0' as a valid ref.

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git tag v1.0 e73141f

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git push origin v1.0
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 16 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (13/13), 972 bytes | 972.00 KiB/s, done.
Total 13 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), completed with 1 local object.
To github.com:shubhasri015-hash/devops.git
 * [new tag]         v1.0 -> v1.0

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git log --oneline
e73141f (HEAD -> feature-branch, tag: v1.0) add reabse1
a3bbf61 added a file
f594517 added fourth file
3ac1788 (origin/feature-branch) Revert "added sample one"
```

```
Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git tag
v1.0

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git ls-remote --tags origin
e73141f0e175cdf8ce3b75d6911c85808a2f2bdd        refs/tags/v1.0

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$
```

8. Clone with Sparse Checkout

Clone only a subdirectory of a repo using sparse checkout

- **Steps :** git clone — Clones a repository.
- --filter=blob:none — Downloads file contents only when needed.
- git sparse-checkout init --cone — Enables sparse checkout.
- git sparse-checkout set — Selects the directory to check out.

  Result: Successfully checked out only the required subdirectory instead of all repository files.

```
Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ cd devops

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main)
$ cd ..

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ git clone --no-checkout --filter=blob:none git@github.com:shubhasri015-hash/de
vops.git
fatal: destination path 'devops' already exists and is not an empty directory.

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (feature-branch)
$ cd devops

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main)
$ git sparse-checkout init --cone

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ git sparse-checkout set folder_name

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ ls

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ git sparse-checkout list
folder_name

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ git sparse-checkout set linux

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ git sparse-checkout set linux

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ ls

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ git sparse-checkout list
linux

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$
```

9. Reset vs Revert Challenge

Demonstrate the difference between git reset --hard and git revert in a repo.

git reset --hard HEAD~1	**Moves the branch back one commit and discards associated changes.**
git revert HEAD	**Creates a new commit that undoes the previous commit.**

Important: git reset --hard can permanently discard uncommitted changes. Avoid using it on shared branches because it rewrites local branch history.

Result: Successfully demonstrated the difference between git reset --hard and git revert.

```
Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ git reset --hard
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Total 5 (delta 0), reused 4 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (5/5), 338 bytes | 338.00 KiB/s, done.
Updating files: 100% (5/5), done.
HEAD is now at ab39fe6 added file

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ echo "First change" > file.txt

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ git add file.txt
warning: in the working copy of 'file.txt', LF will be replaced by CRLF the next time Git touches it

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ git commit -m "First commit"
[main d66d15e] First commit
 1 file changed, 1 insertion(+)
 create mode 100644 file.txt

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ echo "Second change" > file.txt

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ git add file.txt
warning: in the working copy of 'file.txt', LF will be replaced by CRLF the next time Git touches it

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ git commit -m "Second commit"
[main 69d720a] Second commit
 1 file changed, 1 insertion(+), 1 deletion(-)

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ git log --oneline
69d720a (HEAD -> main) Second commit
d66d15e First commit
ab39fe6 (origin/main, origin/HEAD) added file
070e5d1 Merge pull request #1 from shubhasri015-hash/feature-branch
2b72766 added hello file
281fc89 Added g1 and g2 files
4ce15cc Initial commit

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ git reset --hard HEAD~1
HEAD is now at d66d15e First commit

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ echo "Third change" >> file.txt

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ git add file.txt
warning: in the working copy of 'file.txt', LF will be replaced by CRLF the next time Git touches it

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ git commit -m "Third commit"
[main 7453b68] Third commit
 1 file changed, 1 insertion(+)

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ git revert HEAD
[main 0e120bd] Revert "Third commit"
 1 file changed, 1 deletion(-)

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ git log --oneline
0e120bd (HEAD -> main) Revert "Third commit"
7453b68 Third commit
```

```
2b72766 added hello file
281fc89 Added g1 and g2 files
4ce15cc Initial commit

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 3 commits.
  (use "git push" to publish your local commits)

You are in a sparse checkout with 100% of tracked files present.

nothing to commit, working tree clean
```

10. Detached HEAD Challenge

Checkout a specific commit (detached HEAD state) and create a new branch from it

- **git log --oneline — Displays commit history.**
- **git checkout COMMIT_HASH — Checks out a specific commit.**
- **Detached HEAD — HEAD points directly to a commit instead of a branch.**
- **git checkout -b new-branch — Creates a branch from the current commit.**

  Result: Successfully checked out a specific commit in detached HEAD state and created a new branch from it.

```
Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ git log --oneline
0e120bd (HEAD -> main) Revert "Third commit"
7453b68 Third commit
d066d15e First commit
ab39fe6 (origin/main, origin/HEAD) added file
07de5d1 Merge pull request #1 from shubhasri015-hash/feature-branch
2b72766 added hello file
281fc89 Added g1 and g2 files
4ce15cc Initial commit

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (main|SPARSE)
$ git checkout 0e120bd
Note: switching to '0e120bd'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 0e120bd Revert "Third commit"

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops ((0e120bd...))|SPARSE)
$ git status
HEAD detached at 0e120bd
You are in a sparse checkout with 100% of tracked files present.

nothing to commit, working tree clean

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops ((0e120bd...))|SPARSE)
$ git checkout -b new-branch
Switched to a new branch 'new-branch'

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (new-branch|SPARSE)
$ git branch
  main
* new-branch

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (new-branch|SPARSE)
$ git status
On branch new-branch
You are in a sparse checkout with 100% of tracked files present.

nothing to commit, working tree clean

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops/devops (new-branch|SPARSE)
$
```

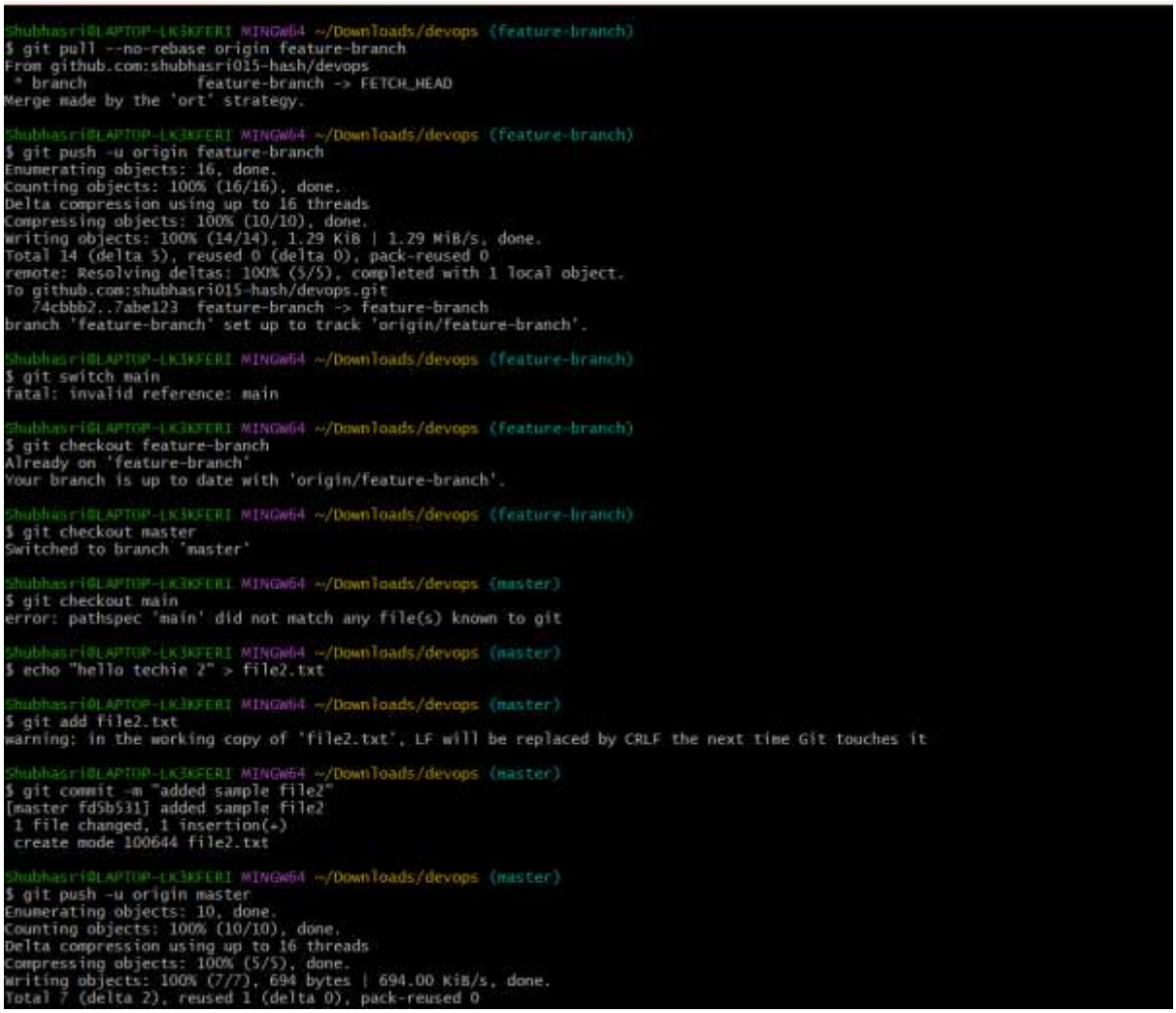
11. Git Hooks Challenge

Configure a pre-commit hook to reject commits without a message format (e.g., must start with JIRA-XXX).

Steps:

- **Git hook: Automatically runs a script during a Git operation.**
- **commit-msg: Checks the commit message before the commit is completed.**
- **grep -qE: Checks whether the message matches the required format.**
- **exit 1: Rejects the commit.**
- **exit 0: Allows the commit.**

Result: Configured a Git hook to reject commit messages that do not follow the required JIRA-XXX format.



12. Squash Merge vs Rebase Merge

Show the difference between squash merge and rebase merge with evidence.

Squash Merge

Two feature commits become one commit on main.

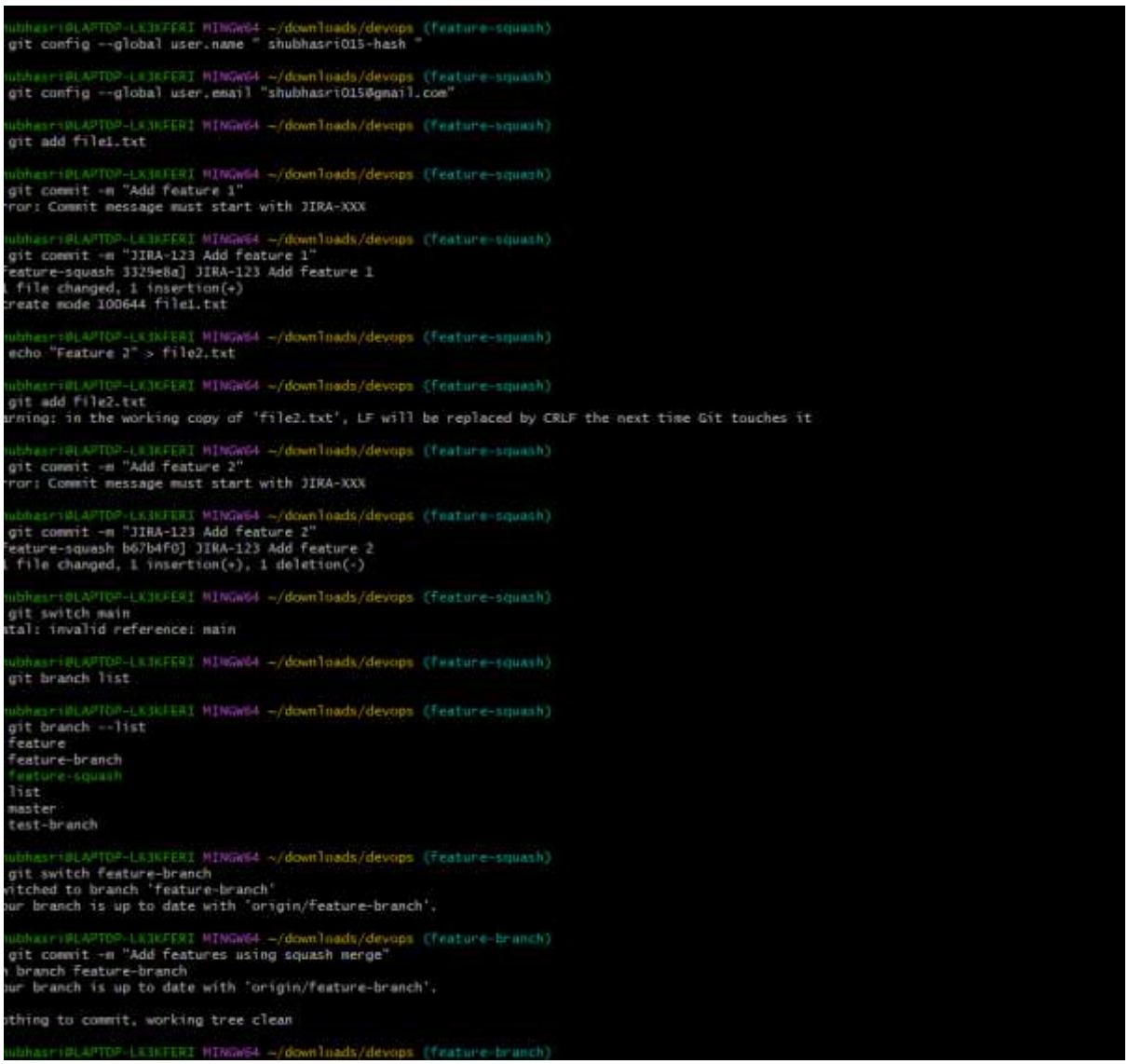
Add features using squash merge

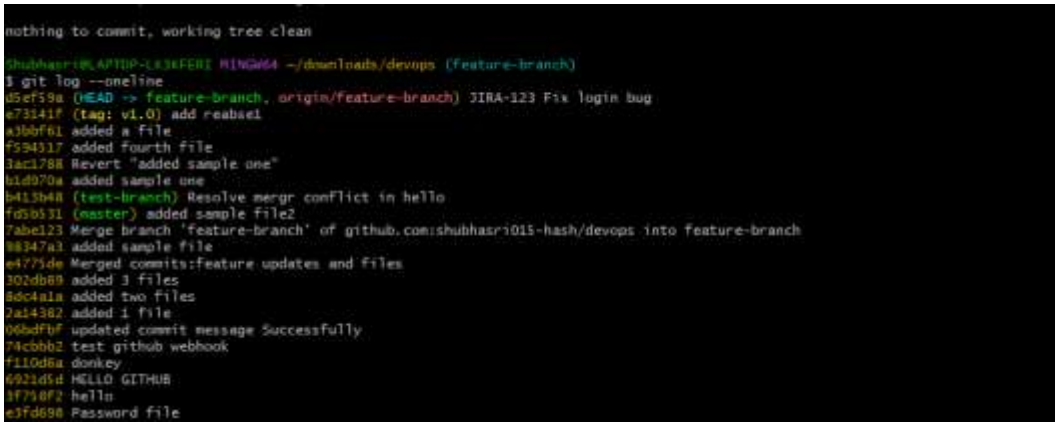
Rebase Merge

Two feature commits remain separate, with a linear history.

Add feature 3 → Add feature 4

Conclusion: Squash merge combines multiple commits into one, whereas rebase merge preserves individual commits and creates a linear history.





```
nothing to commit, working tree clean

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/downloads/devops (feature-branch)
$ git log --oneline
d5ef59a (HEAD -> feature-branch, origin/feature-branch) JIRA-123 Fix login bug
e73141f (tag: v1.0) add reabse1
a3bbf61 added a file
f594317 added fourth file
3ac1788 Revert "added sample one"
b1d070a added sample one
b413b48 (test-branch) Resolve mergr conflict in hello
fd5b531 (master) added sample file2
7abe123 Merge branch 'feature-branch' of github.com:shubhasri015-hash/devops into feature-branch
88347a3 added sample file
e4775de Merged commits:feature updates and files
302db89 added 3 files
8dc4a1a added two files
2a14382 added 1 file
066bdfbf updated commit message Successfully
74cbbb2 test github webhook
f110d6a donkey
6921d5d HELLO GITHUB
3f710f2 hello
e3fd698 Password file
```

```
Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/downloads/devops (feature-branch)
$ git switch -c feature-rebase
Switched to a new branch 'feature-rebase'

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/downloads/devops (feature-rebase)
$ echo "Feature 3" > file3.txt

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/downloads/devops (feature-rebase)
$ git add file3.txt
warning: in the working copy of 'file3.txt', LF will be replaced by CRLF the next time Git touches it

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/downloads/devops (feature-rebase)
$ git commit -m "Jira-123Add feature 3"
Error: Commit message must start with JIRA-XXX

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/downloads/devops (feature-rebase)
$ git commit -m "Jira-123 Add feature 3"
Error: Commit message must start with JIRA-XXX

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/downloads/devops (feature-rebase)
$ git commit -m "JIRA-123 Add feature 3"
[feature-rebase f9d6d2e] JIRA-123 Add feature 3
 1 file changed, 1 insertion(+)
 create mode 100644 file3.txt

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/downloads/devops (feature-rebase)
$ echo "Feature 4" > file4.txt

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/downloads/devops (feature-rebase)
$ git add file4.txt
warning: in the working copy of 'file4.txt', LF will be replaced by CRLF the next time Git touches it

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/downloads/devops (feature-rebase)
$ git commit -m "JIRA-123 Add feature 4"
[feature-rebase 04853db] JIRA-123 Add feature 4
 1 file changed, 1 insertion(+)
 create mode 100644 file4.txt

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/downloads/devops (feature-rebase)
$ git rebase main
fatal: invalid upstream 'main'

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/downloads/devops (feature-rebase)
$ git rebase feature-rebase
Current branch feature-rebase is up to date.

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/downloads/devops (feature-rebase)
$ git switch feature-branch
Switched to branch 'feature-branch'
```

```
Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/downloads/devops (feature-branch)
$ git merge feature-rebase
Updating d5ef59a..04853db
Fast-forward
 file3.txt | 1 +
 file4.txt | 1 +
 2 files changed, 2 insertions(+)
 create mode 100644 file3.txt
 create mode 100644 file4.txt

Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/downloads/devops (feature-branch)
$ git log --oneline
04853db (HEAD -> feature-branch, feature-rebase) JIRA-123 Add feature 4
f9d6d2e JIRA-123 Add feature 3
d5ef59a (origin/feature-branch) JIRA-123 Fix login bug
e73141f (tag: v1.0) add reabse1
a3bbf61 added a file
f594517 added fourth file
```

13. Fork & Pull Request Workflow

Fork a repo, make a change, and submit a pull request to the original repo.

Fork a repo, make a change, and submit a pull request to the original repo.

Fork

Created your copy of your friend's repository.

Make a change

Added a contribution note to README.md.

Commit and push

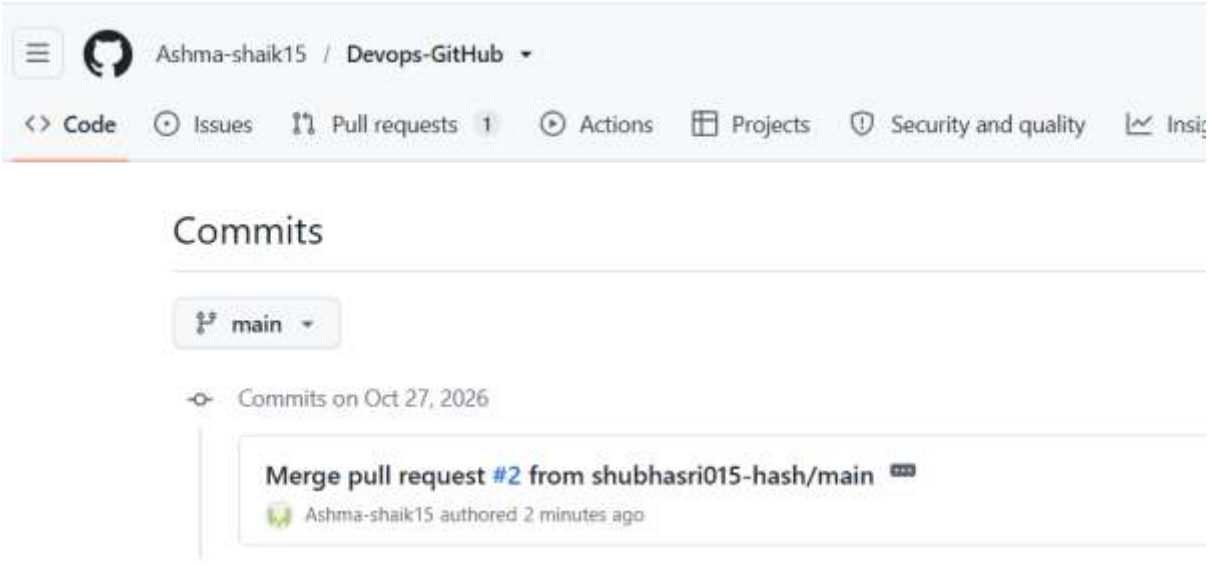
Saved and uploaded your change to GitHub.

Create Pull Request

PR #4 is open — successfully completed.

Review and merge

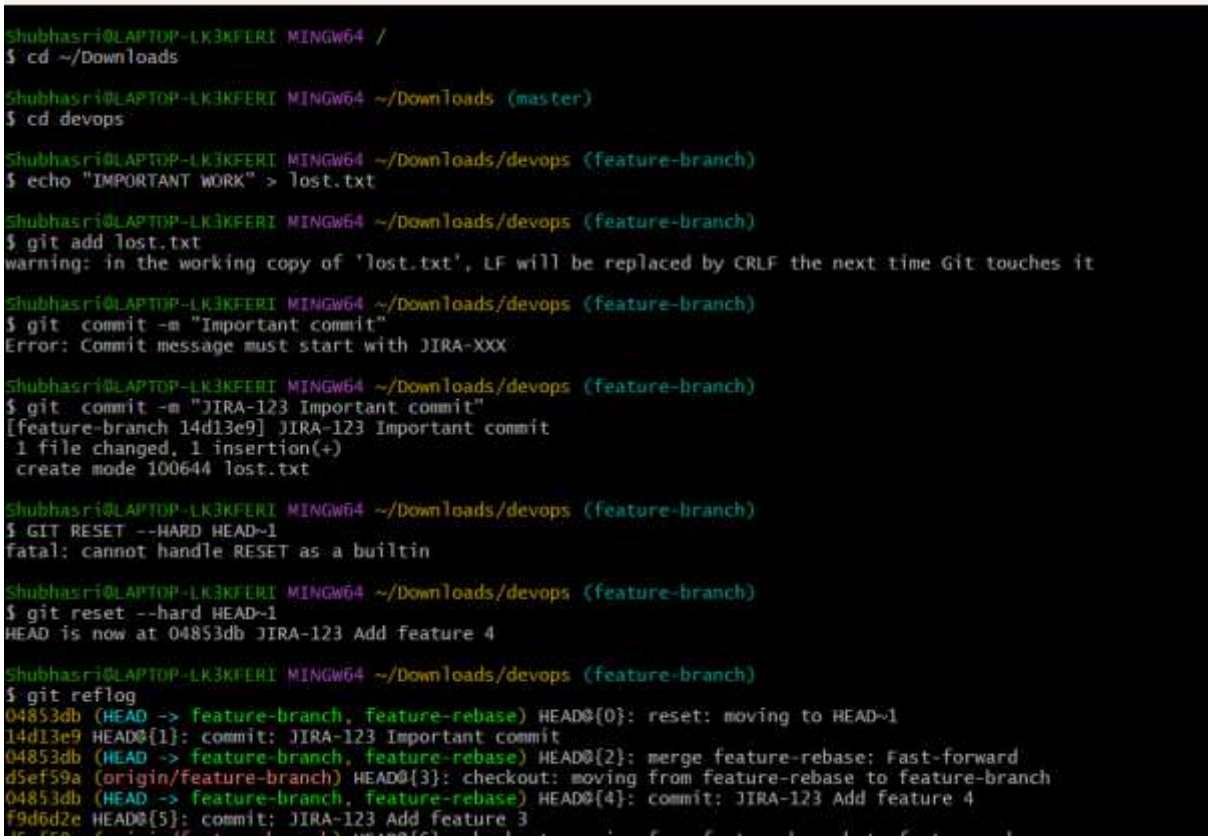
Waiting for your friend to review and merge it.

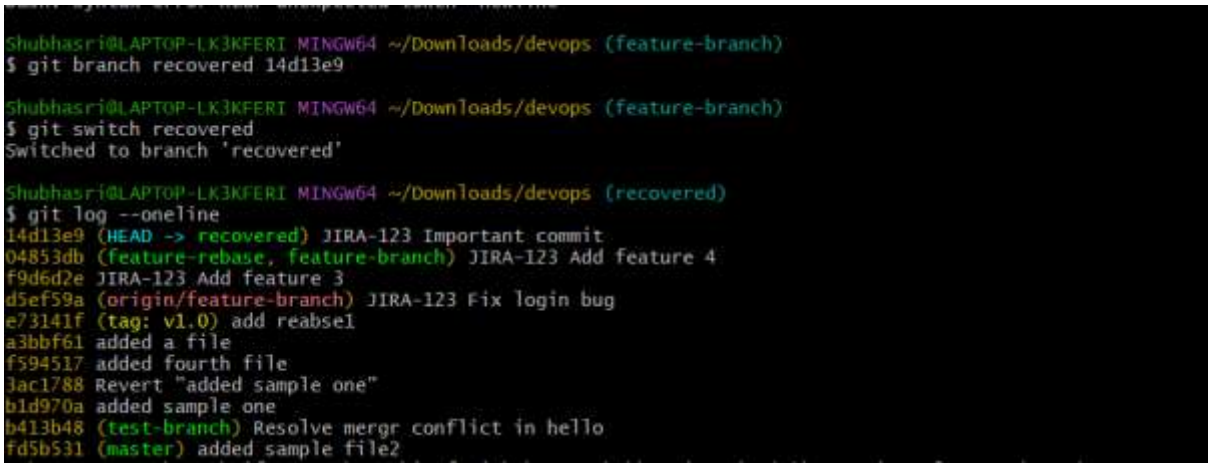


14 . Recover Lost Commit

- **Commit something, reset hard, and then recover it using git reflog**

- **Steps**
- Step 1: Create a file and commit it
- Reset the commit (git reset --hard HEAD~1)
- Check the Git reflog
- Recover the lost commit
- The lost commit is recovered using git reflog.
- Run git reflog before performing other resets, and copy the correct commit hash. git reset --hard can permanently discard uncommitted changes, so use it carefully.






Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (recovered)
$ cat lost.txt
IMPORTANT WORK
Shubhasri@LAPTOP-LK3KFERI MINGW64 ~/Downloads/devops (recovered)
$